

MODUL PRAKTIKUM

MATA KULIAH PEMROGRAMAN MOBILE

PERTEMUAN 02

Fitur, Form Validation & Navigation pada Flutter

SEMESTER GANJIL

TAHUN AJARAN 2026 - 2027



Disusun oleh:

Aina Hubby Aziira, M.Eng

Farhan Fitrahadi

Hafiz Muhammad Faqih

Putri Diva Riyanti

Fachri Akbar

Fathiya Alzhafira

Rahil Akram Hammad

Varissa Anzani Badri

**DEPARTEMEN SISTEM INFORMASI
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026**

IDENTITAS PRAKTIKUM

IDENTITAS MATA KULIAH	
Kode mata kuliah	
Nama mata kuliah	Pemrograman Mobile
CPMK yang dibebankan pada praktikum	
Materi Praktikum Pertemuan 2	1. Dari PRD ke Fitur dan Alur Aplikasi 2. Input Pengguna dan Validasi Form 3. Penanganan Error dan Status Tampilan 4. Navigasi: Navigator, Named Routes, Pengiriman dan Pengembalian Data
Target Praktikum 2	Setiap kelompok mampu mengimplementasikan fitur dan alur navigasi aplikasi Flutter berdasarkan PRD, termasuk user input, validation, error handling, navigation, dan data passing.

IDENTITAS DOSEN DAN ASISTEN MAHASISWA	
Nama Dosen Pengampu	1. Aina Hubby Aziira, M.Eng
Nama Asisten Mahasiswa (Kelas A)	1. 2311521006 - Nayla Thahira Meldian 2. 2311522006 - Siti Aliani Husnah.F 3. 2311522010 - Kezia Valerina Damanik 4. 2311522022 - Laila Qadriyah 5. 2311522024 - Revin Pahlevi 6. 2311522025 - Ervizon Fariz 7. 2311522035 - Hafiz Muhammad Faqih 8. 2311522037 - Farhan Fitrahadi 9. 2311523004 - Fachri Akbar 10. 2311523009 - Fathiya Alzhafira 11. 2311523011 - Muhammad Afiq Jakhel 12. 2311523012 - Rahil Akram Hammad 13. 2311523013 - Putri Diva Riyanti 14. 2311523036 - Varissa Anzani Badri
Nama Asisten Mahasiswa (Kelas B)	1. 2311521006 - Nayla Thahira Meldian 2. 2311522006 - Siti Aliani Husnah.F 3. 2311522010 - Kezia Valerina Damanik 4. 2311522022 - Laila Qadriyah 5. 2311522024 - Revin Pahlevi 6. 2311522025 - Ervizon Fariz 7. 2311522035 - Hafiz Muhammad Faqih

IDENTITAS DOSEN DAN ASISTEN MAHASISWA	
	8. 2311522037 - Farhan Fitrahadi 9. 2311523004 - Fachri Akbar 10. 2311523009 - Fathiya Alzhafira 11. 2311523011 - Muhammad Afiq Jakhel 12. 2311523012 - Rahil Akram Hammad 13. 2311523013 - Putri Diva Riyanti 14. 2311523036 - Varissa Anzani Badri

DAFTAR ISI

PENDAHULUAN	4
A. Tujuan Praktikum.....	4
B. Prasyarat	4
C. Studi Kasus.....	4
BAGIAN 1	6
A. Menurunkan Kebutuhan menjadi Fitur	6
B. Route Map.....	6
C. Model Data dan Repository.....	7
BAGIAN 2	10
A. TextField dan TextFormField	10
B. Membaca Nilai Input.....	10
C. Validasi Form.....	10
D. Waktu Validasi.....	11
E. Validator yang Dapat Digunakan Kembali	12
F. Properti TextFormField yang Sering Digunakan	13
BAGIAN 3	14
A. Proses Asinkron	14
B. Menangani Error dengan try, catch, dan finally	14
C. Status Tampilan.....	14
D. Pemeriksaan mounted	15
E. Menyampaikan Pesan kepada Pengguna.....	15
BAGIAN 4	17
A. Route dan Navigator	17
B. Named Routes	17
C. onGenerateRoute dan onUnknownRoute.....	18
D. Metode Navigator Lainnya	19
E. Mengirim Data ke Layar Lain	20
F. Mengembalikan Data dari Layar	20
G. Mengendalikan Tombol Kembali dengan PopScope.....	21
TROUBLESHOOTING.....	22
REFERENSI	23
LAMPIRAN A.....	25

PENDAHULUAN

A. Tujuan Praktikum

Setelah mengikuti praktikum ini, praktikan diharapkan mampu:

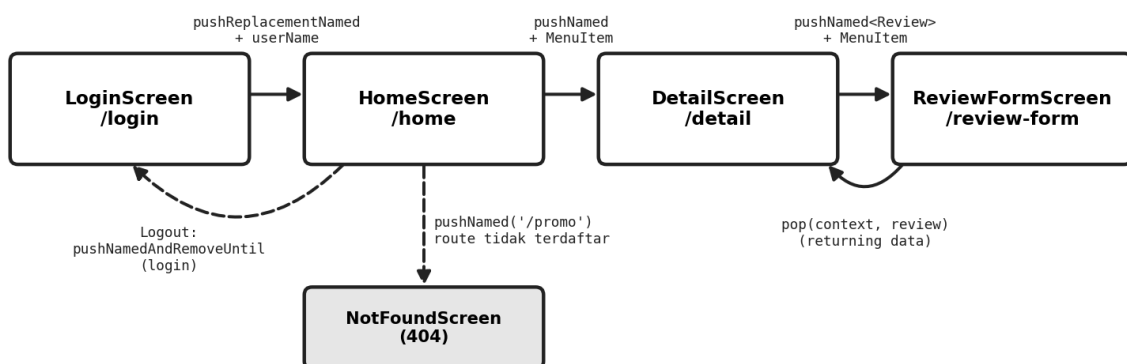
1. menurunkan kebutuhan pada PRD menjadi daftar fitur, alur layar, dan daftar route aplikasi;
2. membuat form input dengan validasi menggunakan Form, TextFormField, dan fungsi validator;
3. menangani proses asinkron dan kesalahan (*error*) serta menampilkan status tampilan yang sesuai;
4. mengimplementasikan perpindahan layar menggunakan Navigator dan *named routes*;
5. mengirim data ke layar lain dan menerima data kembali dari layar lain.

B. Prasyarat

- Lingkungan pengembangan Flutter telah terpasang dan dapat menjalankan aplikasi (Praktikum 1).
- Repository kelompok berisi UI hasil Praktikum 1 dapat di-*clone* dan dijalankan.
- Praktikan telah memahami *widget tree*, *StatelessWidget*, dan *StatefulWidget*.

C. Studi Kasus

Contoh pada modul ini menggunakan studi kasus **Campus Food**, yaitu aplikasi untuk melihat menu kantin kampus. Studi kasus ini melanjutkan contoh pada materi kuliah *Data Modeling, Architecture & Flutter UI*, sehingga model MenuItem yang dirancang pada materi tersebut digunakan kembali. Alur aplikasi ditunjukkan pada Gambar 1.



Gambar 1. Alur layar aplikasi Campus Food

Struktur folder project disusun dengan memisahkan tampilan (screens, widgets), data (models, data), dan pengaturan route (routes).

```
lib/  
├── main.dart  
├── models/      menu_item.dart, review.dart  
├── data/        auth_repository.dart, menu_repository.dart  
├── routes/      app_routes.dart  
├── screens/     login, home, detail, review_form, not_found  
├── utils/       validators.dart  
└── widgets/     menu_card.dart, state_views.dart
```

Kode lengkap seluruh file studi kasus terdapat pada Lampiran A.

BAGIAN 1

DARI PRD KE FITUR DAN ALUR APLIKASI

A. Menurunkan Kebutuhan menjadi Fitur

Materi kuliah menekankan bahwa pembuatan antarmuka dimulai dari kebutuhan (*requirements*) dan alur pengguna (*user flow*), bukan dari pemilihan widget secara acak. Kebutuhan pada PRD dapat dirinci menjadi *user story*, yaitu kebutuhan yang ditulis dari sudut pandang pengguna, dan *acceptance criteria*, yaitu kondisi terukur yang menandakan fitur telah berfungsi dengan benar. *Acceptance criteria* inilah yang kemudian diterjemahkan menjadi aturan validasi, pesan kesalahan, dan alur navigasi.

Tabel 1 menunjukkan contoh penurunan kebutuhan pada studi kasus Campus Food.

Kebutuhan	User story	Acceptance criteria
Login	Sebagai mahasiswa, saya ingin login agar dapat melihat menu kantin.	Email wajib diisi dan berformat benar. Password minimal 8 karakter. Setelah login berhasil, pengguna tidak kembali ke halaman login ketika menekan tombol kembali.
Melihat menu	Sebagai mahasiswa, saya ingin melihat daftar menu per kategori.	Ditampilkan indikator saat data dimuat. Ditampilkan pesan jika data kosong. Ditampilkan pesan dan tombol ulang jika data gagal dimuat.
Menulis ulasan	Sebagai mahasiswa, saya ingin memberi rating dan komentar pada menu.	Rating berupa angka 1 sampai 5. Komentar minimal 10 karakter. Ulasan baru langsung tampil pada halaman detail.

Tabel 1. Contoh user story dan acceptance criteria

B. Route Map

Setiap layar kemudian diberi nama route. Untuk setiap route ditentukan data yang diterima ketika layar dibuka dan data yang dikembalikan ketika layar ditutup. Daftar ini disebut *route map* (Tabel 2) dan menjadi acuan bersama bagi anggota kelompok sebelum penulisan kode.

Route	Layar	Data yang diterima	Data yang dikembalikan
/login	LoginScreen	-	-
/home	HomeScreen	String (nama pengguna)	-

Route	Layar	Data yang diterima	Data yang dikembalikan
/detail	DetailScreen	MenuItem	-
/review-form	ReviewFormScreen	MenuItem	Review

Tabel 2. Route map aplikasi Campus Food

C. Model Data dan Repository

Data yang berpindah antarlayar direpresentasikan dengan kelas model sehingga tipe setiap atribut jelas. Kelas MenuItem berikut mengikuti rancangan pada materi kuliah dan dilengkapi *factory constructor* `fromJson` untuk mengubah data JSON menjadi objek.

lib/models/menu_item.dart

```

/// Model data untuk satu menu makanan/minuman.
/// Struktur ini mengikuti data model yang dirancang pada pertemuan teori.
class MenuItem {
  final String id;
  final String name;
  final double price;
  final String category;
  final String cafeteriaId;
  final String cafeteriaName;
  final String description;

  const MenuItem({
    required this.id,
    required this.name,
    required this.price,
    required this.category,
    required this.cafeteriaId,
    required this.cafeteriaName,
    this.description = '',
  });

  /// Mengubah data JSON (Map) menjadi object MenuItem.
  factory MenuItem.fromJson(Map<String, dynamic> json) {
    return MenuItem(
      id: json['id'] as String,
      name: json['name'] as String,
      price: (json['price'] as num).toDouble(),
      category: json['category'] as String,
      cafeteriaId: json['cafeteriaId'] as String,
      cafeteriaName: json['cafeteriaName'] as String,
      description: json['description'] as String? ?? '',
    );
  }

  /// Format harga: 15000 -> "Rp 15.000"
  String get formattedPrice {
    final digits = price.toStringAsFixed(0);
    final buffer = StringBuffer();
    for (var i = 0; i < digits.length; i++) {
      if (i > 0 && (digits.length - i) % 3 == 0) buffer.write('.');
      buffer.write(digits[i]);
    }
  }
}

```

```

    return 'Rp $buffer';
  }
}

```

lib/models/review.dart

```

/// Model data untuk review yang ditulis mahasiswa pada sebuah menu.
class Review {
  final String id;
  final String menuItemId;
  final String reviewerName;
  final int rating;
  final String comment;

  const Review({
    required this.id,
    required this.menuItemId,
    required this.reviewerName,
    required this.rating,
    required this.comment,
  });
}

```

Karena aplikasi belum terhubung ke *backend*, sumber data dibuat dalam bentuk kelas *repository* yang mengembalikan Future dan mensimulasikan waktu tunggu dengan Future.delayed. Parameter simulateError digunakan untuk menguji kondisi gagal.

lib/data/menu_repository.dart

```

import '../models/menu_item.dart';

/// Data layer: bertugas menyediakan data menu.
/// Untuk praktikum ini data masih lokal (dummy JSON) dan
/// mensimulasikan pengambilan data dari server dengan jeda 2 detik.
class MenuRepository {
  static const List<Map<String, dynamic>> _menuJson = [
    {
      'id': 'M001',
      'name': 'Nasi Goreng',
      'price': 15000,
      'category': 'Rice',
      'cafeteriaId': 'C01',
      'cafeteriaName': 'Kantin Pusat',
      'description': 'Nasi goreng dengan telur mata sapi dan kerupuk.',
    },
    {
      'id': 'M002',
      'name': 'Mie Ayam',
      'price': 13000,
      'category': 'Noodles',
      'cafeteriaId': 'C02',
      'cafeteriaName': 'Kantin FTI',
      'description': 'Mie ayam dengan pangsit dan sawi segar.',
    },
    {
      'id': 'M003',
      'name': 'Es Teh Manis',
      'price': 5000,
      'category': 'Drinks',
      'cafeteriaId': 'C03',
    },
  ],
}

```

```

        'cafeteriaName': 'Kantin Ekonomi',
        'description': 'Teh manis dingin, cocok untuk siang hari.',
    },
    {
        'id': 'M004',
        'name': 'Ayam Geprek',
        'price': 18000,
        'category': 'Rice',
        'cafeteriaId': 'C01',
        'cafeteriaName': 'Kantin Pusat',
        'description': 'Ayam goreng tepung dengan sambal bawang pedas.',
    },
    {
        'id': 'M005',
        'name': 'Bakso',
        'price': 16000,
        'category': 'Noodles',
        'cafeteriaId': 'C03',
        'cafeteriaName': 'Kantin Ekonomi',
        'description': 'Bakso sapi dengan mie kuning dan bihun.',
    },
    {
        'id': 'M006',
        'name': 'Jus Alpukat',
        'price': 10000,
        'category': 'Drinks',
        'cafeteriaId': 'C02',
        'cafeteriaName': 'Kantin FTI',
        'description': 'Jus alpukat dengan susu coklat.',
    },
    ],
];

/// Mengambil daftar menu.
/// [simulateError] = true akan melempar Exception untuk latihan error state.
Future<List<MenuItem>> fetchMenus({bool simulateError = false}) async {
    await Future.delayed(const Duration(seconds: 2));
    if (simulateError) {
        throw Exception('Gagal terhubung ke server. Periksa koneksi internet.');
```

BAGIAN 2

INPUT PENGGUNA DAN VALIDASI FORM

A. TextField dan TextFormField

Flutter menyediakan dua widget kolom teks, yaitu `TextField` dan `TextFormField`. `TextField` merupakan widget input teks yang paling umum digunakan. `TextFormField` membungkus `TextField` dan mengintegrasikannya dengan widget `Form`, sehingga menyediakan fungsi tambahan seperti validasi. Keduanya dapat diberi label, ikon, teks petunjuk, dan teks kesalahan melalui properti `decoration` bertipe `InputDecoration`.

B. Membaca Nilai Input

Nilai yang diketik pengguna dibaca menggunakan `TextEditingController`. Controller dibuat, dihubungkan ke kolom teks melalui properti `controller`, lalu nilainya dibaca melalui properti `text`. Controller harus dibuang dengan memanggil `dispose()` ketika tidak digunakan lagi agar sumber daya yang dipakai dilepaskan.

```
class _LoginScreenState extends State<LoginScreen> {
  final _emailController = TextEditingController();

  @override
  void dispose() {
    _emailController.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return TextFormField(controller: _emailController);
  }
}

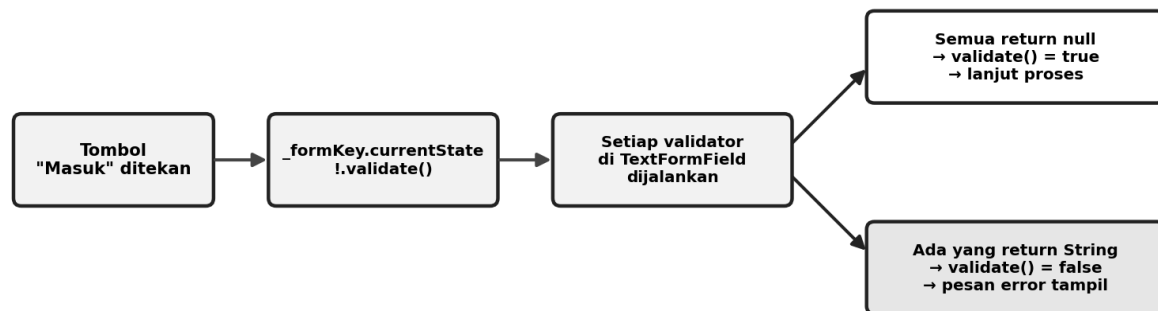
// membaca nilai: _emailController.text
```

C. Validasi Form

Dokumentasi Flutter menjelaskan tiga langkah membuat form dengan validasi:

1. **Membuat Form dengan GlobalKey.** Widget `Form` berfungsi sebagai wadah untuk mengelompokkan dan memvalidasi beberapa kolom input. `GlobalKey<FormState>` memberikan identitas unik pada form dan memungkinkan form divalidasi. Form dibuat di dalam `StatefulWidget` agar key cukup dibuat satu kali, tidak setiap kali method `build` dijalankan.
2. **Menambahkan TextFormField dengan fungsi validator.** Jika input tidak valid, validator mengembalikan `String` berisi pesan kesalahan. Jika tidak ada kesalahan, validator harus mengembalikan `null`.

3. **Membuat tombol untuk memvalidasi dan mengirim form.** Saat tombol ditekan, method `validate()` dipanggil untuk memeriksa seluruh kolom.



Gambar 2. Alur validasi form

```

final _formKey = GlobalKey<FormState>();

Form(
  key: _formKey,
  child: Column(
    children: [
      TextFormField(
        decoration: const InputDecoration(labelText: 'Email'),
        validator: (value) {
          if (value == null || value.isEmpty) {
            return 'Email wajib diisi';
          }
          return null;
        },
      ),
      ElevatedButton(
        onPressed: () {
          if (_formKey.currentState!.validate()) {
            // semua kolom valid, data dapat diproses
          }
        },
        child: const Text('Kirim'),
      ),
    ],
  ),
)

```

D. Waktu Validasi

Secara bawaan, validasi hanya berjalan ketika `validate()` dipanggil. Waktu validasi dapat diatur melalui properti `autovalidateMode` pada `Form` atau `TextFormField` dengan nilai berikut.

Nilai	Keterangan
<code>AutovalidateMode.disabled</code>	Tidak ada validasi otomatis.
<code>AutovalidateMode.always</code>	Validasi otomatis berjalan walaupun belum ada interaksi pengguna.

Nilai	Keterangan
AutovalidateMode.onUserInteraction	Validasi otomatis berjalan setiap kali pengguna berinteraksi dengan kolom.
AutovalidateMode.onUnfocus	Validasi otomatis berjalan setelah kolom kehilangan fokus.

E. Validator yang Dapat Digunakan Kembali

Aturan validasi yang sama sering digunakan pada beberapa layar. Oleh karena itu, aturan tersebut dikumpulkan dalam satu kelas agar tidak ditulis berulang.

lib/utils/validators.dart

```

/// Kumpulan fungsi validasi yang bisa dipakai ulang (reusable).
/// Aturan validator di Flutter:
///   - return null      -> input VALID
///   - return 'pesan'   -> input TIDAK VALID, pesan tampil di bawah field
class Validators {
  Validators._();

  static String? requiredField(String? value, {String fieldName = 'Field ini'}) {
    if (value == null || value.trim().isEmpty) {
      return '$fieldName wajib diisi';
    }
    return null;
  }

  static String? email(String? value) {
    final requiredError = requiredField(value, fieldName: 'Email');
    if (requiredError != null) return requiredError;

    final emailRegex = RegExp(r'^[\w.+-]+@[ \w-]+(\.[ \w-]+)?$');
    if (!emailRegex.hasMatch(value!.trim())) {
      return 'Format email tidak valid';
    }
    return null;
  }

  static String? password(String? value) {
    if (value == null || value.isEmpty) return 'Password wajib diisi';
    if (value.length < 8) return 'Password minimal 8 karakter';
    return null;
  }

  static String? minLength(
    String? value,
    int min, {
    String fieldName = 'Field ini',
  }) {
    final requiredError = requiredField(value, fieldName: fieldName);
    if (requiredError != null) return requiredError;
    if (value!.trim().length < min) {
      return '$fieldName minimal $min karakter';
    }
    return null;
  }
}

```

```

static String? rating(String? value) {
    final requiredError = requiredField(value, fieldName: 'Rating');
    if (requiredError != null) return requiredError;

    final number = int.tryParse(value!.trim());
    if (number == null) return 'Rating harus berupa angka';
    if (number < 1 || number > 5) return 'Rating harus di antara 1 sampai 5';
    return null;
}
}

```

Validator dipasang langsung apabila parameternya sesuai, misalnya validator: `Validators.email`, atau dibungkus fungsi apabila memerlukan parameter tambahan, misalnya validator: `(value) => Validators.minLength(value, 3, fieldName: 'Nama')`.

F. Properti TextFormField yang Sering Digunakan

Properti	Fungsi
<code>keyboardType</code>	Menentukan jenis keyboard, misalnya <code>TextInputType.emailAddress</code> atau <code>TextInputType.number</code> .
<code>textInputAction</code>	Menentukan tombol aksi pada keyboard, misalnya <code>TextInputAction.next</code> atau <code>TextInputAction.done</code> .
<code>obscureText</code>	Menyembunyikan teks yang diketik, digunakan untuk password.
<code>enabled</code>	Mengaktifkan atau menonaktifkan kolom.
<code>maxLength</code>	Membatasi jumlah karakter.

BAGIAN 3

PENANGANAN ERROR DAN STATUS TAMPILAN

A. Proses Asinkron

Proses yang membutuhkan waktu, seperti mengambil data dari server, dijalankan secara asinkron dan menghasilkan objek Future. Pada Dart, hasil Future ditunggu menggunakan kata kunci `await` di dalam fungsi yang ditandai `async`.

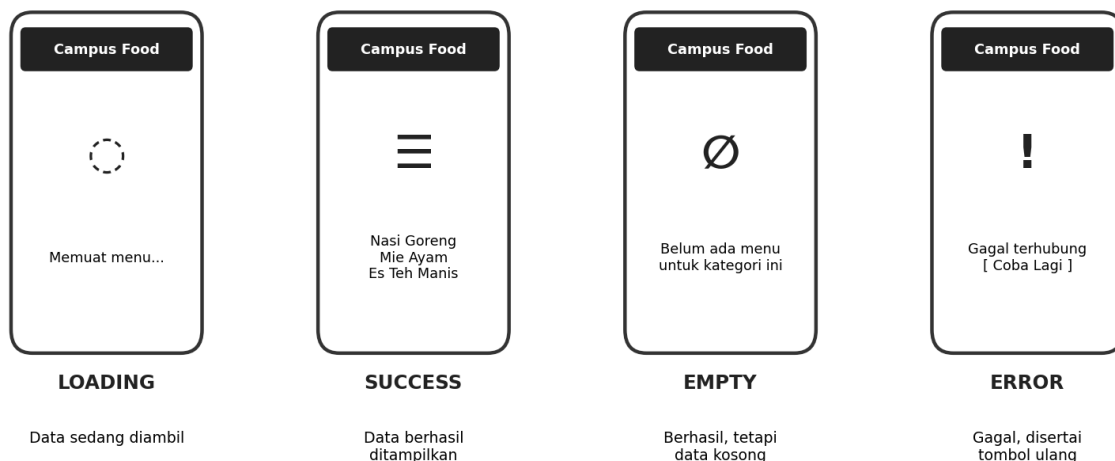
B. Menangani Error dengan `try`, `catch`, dan `finally`

Kesalahan yang terjadi selama proses berjalan dilempar sebagai *exception*. Exception ditangkap menggunakan blok `try-catch`; klausa `on` digunakan untuk menangkap tipe exception tertentu. Blok `finally` selalu dijalankan, baik proses berhasil maupun gagal, sehingga tepat digunakan untuk mengakhiri indikator loading.

```
Future<void> _submit() async {
  setState(() => _isLoading = true);
  try {
    final userName = await _authRepository.login(
      email: email,
      password: password,
    );
    // proses berhasil
  } on AuthException catch (e) {
    setState(() => _errorMessage = e.message); // exception yang dikenali
  } catch (e) {
    setState(() => _errorMessage = 'Terjadi kesalahan. Coba lagi nanti.');
```

C. Status Tampilan

Dokumentasi widget `FutureBuilder` menggambarkan bahwa tampilan yang bergantung pada Future dibangun ulang sesuai kondisi Future tersebut: masih menunggu (`ConnectionState.waiting`), selesai dengan data, atau selesai dengan error. Studi kasus ini menerapkan pola yang sama secara eksplisit menggunakan enum dan `setState`, ditambah satu kondisi ketika data berhasil dimuat tetapi kosong (Gambar 3).



Gambar 3. Status tampilan pada layar yang memuat data

```
enum ViewStatus { loading, success, error }

Widget _buildContent() {
  return switch (_status) {
    ViewStatus.loading => const LoadingView(message: 'Memuat menu...'),
    ViewStatus.error => ErrorView(message: _errorMessage, onRetry: _loadMenus),
    ViewStatus.success => _buildMenuList(), // menampilkan EmptyView jika list kosong
  };
}
```

Widget `LoadingView`, `EmptyView`, dan `ErrorView` terdapat pada `lib/widgets/state_views.dart` (Lampiran A).

D. Pemeriksaan *mounted*

Objek `State` berada dalam kondisi *mounted* sejak dikaitkan dengan `BuildContext` sampai method `dispose` dipanggil. Memanggil `setState` ketika `mounted` bernilai `false` merupakan kesalahan. Selama menunggu `await`, layar dapat saja telah ditutup oleh pengguna. Oleh karena itu, nilai `mounted` diperiksa setelah `await` sebelum menggunakan `setState` atau `context`. Aturan *linter* Dart `use_build_context_synchronously` memberikan peringatan apabila `BuildContext` digunakan setelah jeda asinkron tanpa pemeriksaan tersebut.

```
final userName = await _authRepository.login(...);
if (!mounted) return;
Navigator.pushReplacementNamed(context, AppRoutes.home, arguments: userName);
```

E. Menyampaikan Pesan kepada Pengguna

Widget	Keterangan
<code>errorText</code> pada <code>TextFormField</code>	Teks kesalahan di bawah kolom, dihasilkan dari validator.

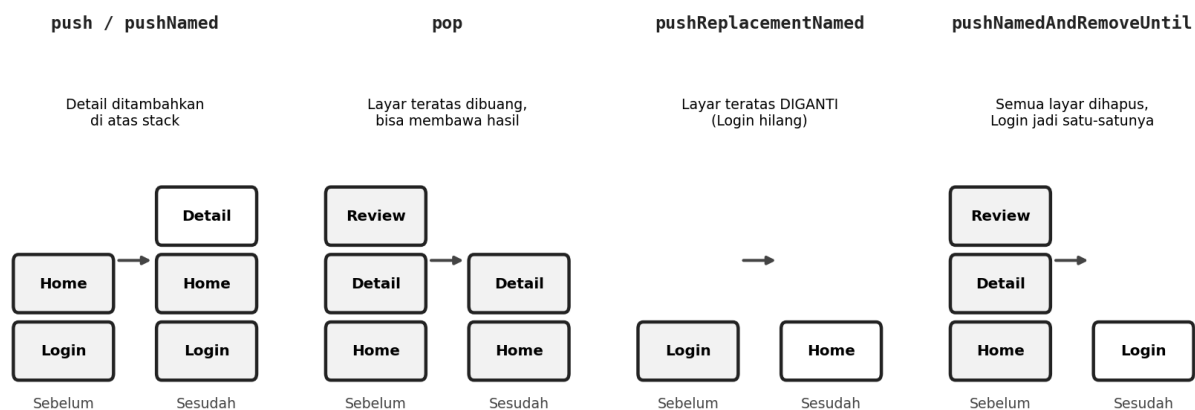
Widget	Keterangan
SnackBar	Pesan ringan yang tampil sesaat di bagian bawah layar, dapat disertai satu aksi.
AlertDialog	Dialog yang menginformasikan situasi yang memerlukan tanggapan pengguna.

BAGIAN 4

NAVIGASI

A. Route dan Navigator

Pada Flutter, layar atau halaman disebut *route*. Widget Navigator menampilkan layar dalam bentuk tumpukan (*stack*) dengan animasi transisi yang sesuai dengan platform. Method `push()` menambahkan route ke tumpukan, sedangkan `pop()` menghapus route teratas. `MaterialPageRoute` adalah turunan Route yang menggunakan animasi transisi Material Design.



Gambar 4. Perubahan tumpukan route pada beberapa operasi Navigator

```
// membuka layar baru
Navigator.push(
  context,
  MaterialPageRoute(builder: (context) => const SecondScreen()),
);

// kembali ke layar sebelumnya
Navigator.pop(context);
```

B. Named Routes

Apabila layar yang sama dibuka dari banyak bagian aplikasi, pembuatan route secara langsung dapat menimbulkan duplikasi kode. Solusinya adalah mendefinisikan *named route* dan berpindah layar menggunakan `Navigator.pushNamed()`. Properti `initialRoute` pada `MaterialApp` menentukan route awal, sedangkan properti `routes` mendefinisikan route yang tersedia beserta widget yang dibangun.

```
MaterialApp(  
  initialRoute: '/',  
  routes: {  
    '/': (context) => const FirstScreen(),  
    '/second': (context) => const SecondScreen(),  
  },  
);  
  
// berpindah layar  
Navigator.pushNamed(context, '/second');
```

Perhatian

Jika menggunakan `initialRoute`, properti `home` tidak boleh didefinisikan.

Catatan

Dokumentasi resmi Flutter tidak merekomendasikan `named routes` untuk sebagian besar aplikasi, karena perilaku *deep link* tidak dapat dikustomisasi dan tombol *forward* pada browser tidak didukung. Sebagai gantinya disarankan `Navigator` dengan `MaterialPageRoute` atau package routing seperti `go_router`. `Named routes` tetap dipelajari pada praktikum ini sebagai dasar konsep `route`, argumen, dan hasil navigasi.

C. `onGenerateRoute` dan `onUnknownRoute`

Selain `routes`, `MaterialApp` menyediakan `onGenerateRoute`, yaitu fungsi yang dipanggil ketika aplikasi berpindah ke `named route` yang tidak terdapat pada `routes`. Fungsi ini menerima `RouteSettings` yang berisi nama `route` dan argumen, lalu mengembalikan `Route`. Apabila `onGenerateRoute` gagal menghasilkan `route`, `MaterialApp` memanggil `onUnknownRoute`, yang umumnya digunakan untuk menampilkan halaman "tidak ditemukan".

```

class AppRoutes {
  static const String login = '/login';
  static const String home = '/home';

  static Route<dynamic>? onGenerateRoute(RouteSettings settings) {
    switch (settings.name) {
      case login:
        return MaterialPageRoute<void>(builder: (_) => const LoginScreen());
      case home:
        final userName = settings.arguments as String? ?? 'Mahasiswa';
        return MaterialPageRoute<void>(
          builder: (_) => HomeScreen(userName: userName),
        );
      default:
        return null; // onUnknownRoute akan dipanggil
    }
  }

  static Route<dynamic> onUnknownRoute(RouteSettings settings) {
    return MaterialPageRoute<void>(
      builder: (_) => NotFoundScreen(routeName: settings.name),
    );
  }
}

// main.dart
MaterialApp(
  initialRoute: AppRoutes.login,
  onGenerateRoute: AppRoutes.onGenerateRoute,
  onUnknownRoute: AppRoutes.onUnknownRoute,
);

```

D. Metode Navigator Lainnya

Metode	Keterangan
pushReplacement / pushReplacementNamed	Mengganti route teratas dengan route baru.
pushAndRemoveUntil / pushNamedAndRemoveUntil	Menambahkan route, lalu menghapus route di bawahnya sampai suatu kondisi terpenuhi.
popUntil	Menghapus route teratas secara berulang sampai suatu kondisi terpenuhi.

Pada studi kasus, pushReplacementNamed digunakan setelah login berhasil agar halaman login tidak tersisa di tumpukan, sedangkan pushNamedAndRemoveUntil digunakan saat logout untuk menghapus seluruh route sebelumnya.

```

Navigator.pushReplacementNamed(context, AppRoutes.home, arguments: userName);

Navigator.pushNamedAndRemoveUntil(context, AppRoutes.login, (route) => false);

```

E. Mengirim Data ke Layar Lain

Data dapat dikirim ke layar tujuan melalui constructor, atau melalui parameter arguments pada `Navigator.pushNamed()`. Argumen dibaca di layar tujuan menggunakan `ModalRoute.of()`, atau dibaca di dalam `onGenerateRoute` lalu diteruskan ke constructor layar.

```
// 1. melalui constructor
Navigator.push(
  context,
  MaterialPageRoute(builder: (context) => DetailScreen(menuItem: menu)),
);

// 2. melalui arguments, dibaca dengan ModalRoute.of()
Navigator.pushNamed(context, '/detail', arguments: menu);
final menu = ModalRoute.of(context)!.settings.arguments as MenuItem;

// 3. melalui arguments, dibaca di onGenerateRoute (digunakan pada studi kasus)
case detail:
  final args = settings.arguments;
  if (args is MenuItem) {
    return MaterialPageRoute<void>(
      builder: (_) => DetailScreen(menuItem: args),
    );
  }
  return null;
```

F. Mengembalikan Data dari Layar

Method `Navigator.pop()` menerima argumen kedua yang opsional, yaitu `result`. Nilai tersebut dikembalikan ke `Future` yang dihasilkan oleh `push` atau `pushNamed` pada layar sebelumnya. Jika layar ditutup tanpa hasil, nilai yang diterima adalah `null`.

```
// DetailScreen: membuka form dan menunggu hasilnya
final newReview = await Navigator.pushNamed<Review>(
  context,
  AppRoutes.reviewForm,
  arguments: widget.menuItem,
);
if (!mounted || newReview == null) return;
setState(() => _reviews.insert(0, newReview));

// ReviewFormScreen: menutup layar sambil mengirim hasil
Navigator.pop(context, review);
```

Perhatian

Apabila memanggil `pushNamed<Review>`, route pada `onGenerateRoute` harus dibuat sebagai `MaterialPageRoute<Review>`. Tipe route yang berbeda menyebabkan *type error* saat aplikasi dijalankan.

G. Mengendalikan Tombol Kembali dengan PopScope

Widget PopScope digunakan untuk mengendalikan navigasi kembali. Properti canPop bernilai false mencegah route ditutup, dan callback onPopInvokedWithResult dipanggil setiap kali terjadi upaya kembali. PopScope menggantikan WillPopScope yang sudah tidak digunakan lagi (*deprecated*) untuk mendukung fitur *predictive back* pada Android 14.

```
PopScope<Object?>(  
  canPop: !_hasUnsavedInput,  
  onPopInvokedWithResult: (didPop, result) async {  
    if (didPop) return;  
    final shouldLeave = await _confirmDiscard();  
    if (shouldLeave && mounted) Navigator.pop(context);  
  },  
  child: Scaffold(...),  
)
```

TROUBLESHOOTING

Pesan / gejala	Penyebab	Solusi
Could not find a generator for route ...	Route tidak terdaftar pada routes/onGenerateRoute dan onUnknownRoute tidak disediakan.	Daftarkan route dan sediakan onUnknownRoute.
Navigator operation requested with a context that does not include a Navigator.	context yang digunakan berada di atas MaterialApp.	Panggil navigasi dari widget di bawah MaterialApp.
setState() called after dispose()	Proses asinkron selesai setelah layar ditutup.	Periksa mounted sebelum setState.
Peringatan use_build_context_synchronously	BuildContext digunakan setelah await.	Tambahkan if (!mounted) return; setelah await.
type 'Null' is not a subtype of type 'MenuItem' in type cast	Argumen tidak dikirim, tetapi di-cast secara paksa.	Kirim arguments dan periksa tipenya dengan is.
type 'MaterialPageRoute<void>' is not a subtype of type 'Route<Review?>?'	Tipe route tidak sesuai dengan tipe hasil pushNamed<Review>.	Gunakan MaterialPageRoute<Review>.
Null check operator used on a null value pada _formKey.currentState!	GlobalKey belum dipasang pada Form.	Tambahkan key: _formKey pada Form.
Validator tidak berjalan	Menggunakan TextField atau kolom berada di luar Form.	Gunakan TextFormField di dalam Form.
A RenderFlex overflowed by ... pixels on the bottom. saat keyboard muncul	Isi Column melebihi ruang layar.	Bungkus dengan SingleChildScrollView.
Perubahan route tidak berpengaruh	Hot reload tidak menjalankan ulang main().	Lakukan hot restart.
'WillPopScope' is deprecated	Menggunakan API lama.	Ganti dengan PopScope.

REFERENSI

- Dart. (t.t.-a). *Asynchronous programming: futures, async, await*. Diakses pada 22 September 2026, dari <https://dart.dev/libraries/async/async-await>
- Dart. (t.t.-b). *Error handling*. Diakses pada 22 September 2026, dari <https://dart.dev/language/error-handling>
- Dart. (t.t.-c). *use_build_context_synchronously*. Diakses pada 22 September 2026, dari https://dart.dev/tools/linter-rules/use_build_context_synchronously
- Departemen Sistem Informasi Universitas Andalas. (2026, 8 September). *Module 3: Data modeling, architecture & Flutter UI* [Materi kuliah Mobile Application Development]. Fakultas Teknologi Informasi, Universitas Andalas.
- Flutter. (t.t.-a). *AlertDialog class*. Diakses pada 22 September 2026, dari <https://api.flutter.dev/flutter/material/AlertDialog-class.html>
- Flutter. (t.t.-b). *Android predictive back*. Diakses pada 22 September 2026, dari <https://docs.flutter.dev/release/breaking-changes/android-predictive-back>
- Flutter. (t.t.-c). *AutovalidateMode enum*. Diakses pada 22 September 2026, dari <https://api.flutter.dev/flutter/widgets/AutovalidateMode.html>
- Flutter. (t.t.-d). *Build a form with validation*. Diakses pada 22 September 2026, dari <https://docs.flutter.dev/cookbook/forms/validation>
- Flutter. (t.t.-e). *Create and style a text field*. Diakses pada 22 September 2026, dari <https://docs.flutter.dev/cookbook/forms/text-input>
- Flutter. (t.t.-f). *FutureBuilder class*. Diakses pada 22 September 2026, dari <https://api.flutter.dev/flutter/widgets/FutureBuilder-class.html>
- Flutter. (t.t.-g). *mounted property*. Diakses pada 22 September 2026, dari <https://api.flutter.dev/flutter/widgets/State/mounted.html>
- Flutter. (t.t.-h). *Navigate to a new screen and back*. Diakses pada 22 September 2026, dari <https://docs.flutter.dev/cookbook/navigation/navigation-basics>
- Flutter. (t.t.-i). *Navigate with named routes*. Diakses pada 22 September 2026, dari <https://docs.flutter.dev/cookbook/navigation/named-routes>
- Flutter. (t.t.-j). *Navigation and routing*. Diakses pada 22 September 2026, dari <https://docs.flutter.dev/ui/navigation>
- Flutter. (t.t.-k). *Pass arguments to a named route*. Diakses pada 22 September 2026, dari <https://docs.flutter.dev/cookbook/navigation/navigate-with-arguments>

Flutter. (t.t.-l). *PopScope class*. Diakses pada 22 September 2026, dari <https://api.flutter.dev/flutter/widgets/PopScope-class.html>

Flutter. (t.t.-m). *Retrieve the value of a text field*. Diakses pada 22 September 2026, dari <https://docs.flutter.dev/cookbook/forms/retrieve-input>

Flutter. (t.t.-n). *Return data from a screen*. Diakses pada 22 September 2026, dari <https://docs.flutter.dev/cookbook/navigation/returning-data>

Flutter. (t.t.-o). *Send data to a new screen*. Diakses pada 22 September 2026, dari <https://docs.flutter.dev/cookbook/navigation/passing-data>

Flutter. (t.t.-p). *SnackBar class*. Diakses pada 22 September 2026, dari <https://api.flutter.dev/flutter/material/SnackBar-class.html>

Flutter. (t.t.-q). *WidgetsApp class*. Diakses pada 22 September 2026, dari <https://api.flutter.dev/flutter/widgets/WidgetsApp-class.html>

LAMPIRAN A

KODE LENGKAP STUDI KASUS CAMPUS FOOD

A.1 lib/main.dart

```
import 'package:flutter/material.dart';

import 'routes/app_routes.dart';

void main() {
  runApp(const CampusFoodApp());
}

class CampusFoodApp extends StatelessWidget {
  const CampusFoodApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Campus Food',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.green),
        useMaterial3: true,
      ),
      // Layar pertama yang dibuka saat aplikasi dijalankan
      initialRoute: AppRoutes.login,
      // Semua route bernama "diterjemahkan" menjadi screen di sini
      onGenerateRoute: AppRoutes.onGenerateRoute,
      // Dipanggil jika onGenerateRoute mengembalikan null (route tidak dikenal)
      onUnknownRoute: AppRoutes.onUnknownRoute,
    );
  }
}
```

A.2 lib/models/menu_item.dart

```
/// Model data untuk satu menu makanan/minuman.
/// Struktur ini mengikuti data model yang dirancang pada pertemuan teori.
class MenuItem {
  final String id;
  final String name;
  final double price;
  final String category;
  final String cafeteriaId;
  final String cafeteriaName;
  final String description;

  const MenuItem({
    required this.id,
    required this.name,
    required this.price,
    required this.category,
    required this.cafeteriaId,
    required this.cafeteriaName,
    this.description = '',
  });

  /// Mengubah data JSON (Map) menjadi object MenuItem.
  factory MenuItem.fromJson(Map<String, dynamic> json) {
```

```

    return MenuItem(
      id: json['id'] as String,
      name: json['name'] as String,
      price: (json['price'] as num).toDouble(),
      category: json['category'] as String,
      cafeteriaId: json['cafeteriaId'] as String,
      cafeteriaName: json['cafeteriaName'] as String,
      description: json['description'] as String? ?? '',
    );
  }

  /// Format harga: 15000 -> "Rp 15.000"
  String get formattedPrice {
    final digits = price.toStringAsFixed(0);
    final buffer = StringBuffer();
    for (var i = 0; i < digits.length; i++) {
      if (i > 0 && (digits.length - i) % 3 == 0) buffer.write('.');
      buffer.write(digits[i]);
    }
    return 'Rp $buffer';
  }
}

```

A.3 lib/models/review.dart

```

/// Model data untuk review yang ditulis mahasiswa pada sebuah menu.
class Review {
  final String id;
  final String menuItemId;
  final String reviewerName;
  final int rating;
  final String comment;

  const Review({
    required this.id,
    required this.menuItemId,
    required this.reviewerName,
    required this.rating,
    required this.comment,
  });
}

```

A.4 lib/data/auth_repository.dart

```
/// Exception khusus untuk kegagalan login.
class AuthException implements Exception {
  final String message;
  const AuthException(this.message);

  @override
  String toString() => message;
}

/// Data layer untuk autentikasi (masih simulasi, belum memakai backend).
class AuthRepository {
  static const String demoEmail = 'mahasiswa@unand.ac.id';
  static const String demoPassword = 'rahasia123';

  /// Mengembalikan nama pengguna jika login berhasil,
  /// atau melempar [AuthException] jika gagal.
  Future<String> login({
    required String email,
    required String password,
  }) async {
    await Future.delayed(const Duration(seconds: 2));
    if (email != demoEmail || password != demoPassword) {
      throw const AuthException('Email atau password salah.');
```

A.5 lib/data/menu_repository.dart

```
import '../models/menu_item.dart';

/// Data layer: bertugas menyediakan data menu.
/// Untuk praktikum ini data masih lokal (dummy JSON) dan
/// mensimulasikan pengambilan data dari server dengan jeda 2 detik.
class MenuRepository {
  static const List<Map<String, dynamic>> _menuJson = [
    {
      'id': 'M001',
      'name': 'Nasi Goreng',
      'price': 15000,
      'category': 'Rice',
      'cafeteriaId': 'C01',
      'cafeteriaName': 'Kantin Pusat',
      'description': 'Nasi goreng dengan telur mata sapi dan kerupuk.',
    },
    {
      'id': 'M002',
      'name': 'Mie Ayam',
      'price': 13000,
      'category': 'Noodles',
      'cafeteriaId': 'C02',
      'cafeteriaName': 'Kantin FTI',
      'description': 'Mie ayam dengan pangsit dan sawi segar.',
    },
    {
      'id': 'M003',
      'name': 'Es Teh Manis',
      'price': 5000,
      'category': 'Drinks',
```

```

        'cafeteriaId': 'C03',
        'cafeteriaName': 'Kantin Ekonomi',
        'description': 'Teh manis dingin, cocok untuk siang hari.',
    },
    {
        'id': 'M004',
        'name': 'Ayam Geprek',
        'price': 18000,
        'category': 'Rice',
        'cafeteriaId': 'C01',
        'cafeteriaName': 'Kantin Pusat',
        'description': 'Ayam goreng tepung dengan sambal bawang pedas.',
    },
    {
        'id': 'M005',
        'name': 'Bakso',
        'price': 16000,
        'category': 'Noodles',
        'cafeteriaId': 'C03',
        'cafeteriaName': 'Kantin Ekonomi',
        'description': 'Bakso sapi dengan mie kuning dan bihun.',
    },
    {
        'id': 'M006',
        'name': 'Jus Alpukat',
        'price': 10000,
        'category': 'Drinks',
        'cafeteriaId': 'C02',
        'cafeteriaName': 'Kantin FTI',
        'description': 'Jus alpukat dengan susu coklat.',
    },
    ],
];

/// Mengambil daftar menu.
/// [simulateError] = true akan melempar Exception untuk latihan error state.
Future<List<MenuItem>> fetchMenus({bool simulateError = false}) async {
    await Future.delayed(const Duration(seconds: 2));
    if (simulateError) {
        throw Exception('Gagal terhubung ke server. Periksa koneksi internet.');
```

A.6 lib/utlis/validators.dart

```

/// Kumpulan fungsi validasi yang bisa dipakai ulang (reusable).
/// Aturan validator di Flutter:
///   - return null      -> input VALID
///   - return 'pesan'   -> input TIDAK VALID, pesan tampil di bawah field
class Validators {
    Validators._();

    static String? requiredField(String? value, {String fieldName = 'Field ini'}) {
        if (value == null || value.trim().isEmpty) {
            return '$fieldName wajib diisi';
        }
        return null;
    }

    static String? email(String? value) {
        final requiredError = requiredField(value, fieldName: 'Email');
```

```

    if (requiredError != null) return requiredError;

    final emailRegex = RegExp(r'^[\w.+-]+@[ \w-]+(\.[ \w-]+)+$');
    if (!emailRegex.hasMatch(value!.trim())) {
        return 'Format email tidak valid';
    }
    return null;
}

static String? password(String? value) {
    if (value == null || value.isEmpty) return 'Password wajib diisi';
    if (value.length < 8) return 'Password minimal 8 karakter';
    return null;
}

static String? minLength(
    String? value,
    int min, {
    String fieldName = 'Field ini',
}) {
    final requiredError = requiredField(value, fieldName: fieldName);
    if (requiredError != null) return requiredError;
    if (value!.trim().length < min) {
        return '$fieldName minimal $min karakter';
    }
    return null;
}

static String? rating(String? value) {
    final requiredError = requiredField(value, fieldName: 'Rating');
    if (requiredError != null) return requiredError;

    final number = int.tryParse(value!.trim());
    if (number == null) return 'Rating harus berupa angka';
    if (number < 1 || number > 5) return 'Rating harus di antara 1 sampai 5';
    return null;
}
}

```

A.7 lib/routes/app_routes.dart

```

import 'package:flutter/material.dart';

import '../models/menu_item.dart';
import '../models/review.dart';
import '../screens/detail_screen.dart';
import '../screens/home_screen.dart';
import '../screens/login_screen.dart';
import '../screens/not_found_screen.dart';
import '../screens/review_form_screen.dart';

/// Satu tempat untuk seluruh nama route dan cara membangun screen-nya.
class AppRoutes {
    AppRoutes._();

    // 1. Nama route disimpan sebagai konstanta (hindari typo string)
    static const String login = '/login';
    static const String home = '/home';
    static const String detail = '/detail';
    static const String reviewForm = '/review-form';
}

```

```

// 2. Menerjemahkan RouteSettings (nama + arguments) menjadi Route
static Route<dynamic>? onGenerateRoute(RouteSettings settings) {
    switch (settings.name) {
        case login:
            return MaterialPageRoute<void>(
                builder: (_) => const LoginScreen(),
                settings: settings,
            );

        case home:
            // Argument berupa String (nama pengguna), boleh kosong
            final userName = settings.arguments as String? ?? 'Mahasiswa';
            return MaterialPageRoute<void>(
                builder: (_) => HomeScreen(userName: userName),
                settings: settings,
            );

        case detail:
            // Argument WAJIB berupa MenuItem -> cek tipenya dulu
            final args = settings.arguments;
            if (args is MenuItem) {
                return MaterialPageRoute<void>(
                    builder: (_) => DetailScreen(menuItem: args),
                    settings: settings,
                );
            }
            return _errorRoute<void>(settings, 'Data menu tidak dikirim ke halaman detail.');
```

```

        case reviewForm:
            final args = settings.arguments;
            if (args is MenuItem) {
                // <Review> = tipe data yang akan DIKEMBALIKAN screen ini saat pop
                return MaterialPageRoute<Review>(
                    builder: (_) => ReviewFormScreen(menuItem: args),
                    settings: settings,
                );
            }
            // Tipe error route disamakan (<Review>) agar tidak terjadi type error
            return _errorRoute<Review>(settings, 'Data menu tidak dikirim ke form review.');
```

```

        default:
            // null -> MaterialApp akan memanggil onUnknownRoute
            return null;
    }
}

// 3. Halaman cadangan untuk route yang tidak terdaftar
static Route<dynamic> onUnknownRoute(RouteSettings settings) {
    return MaterialPageRoute<void>(
        builder: (_) => NotFoundScreen(routeName: settings.name),
        settings: settings,
    );
}

static Route<T> _errorRoute<T>(RouteSettings settings, String message) {
    return MaterialPageRoute<T>(
        builder: (_) => NotFoundScreen(routeName: settings.name, message: message),
        settings: settings,
    );
}
}
}

```

A.8 lib/widgets/state_views.dart

```
import 'package:flutter/material.dart';

/// Tampilan saat data sedang dimuat.
class LoadingView extends StatelessWidget {
  final String message;

  const LoadingView({super.key, this.message = 'Memuat data...'});

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          const CircularProgressIndicator(),
          const SizedBox(height: 16),
          Text(message),
        ],
      ),
    );
  }
}

/// Tampilan saat data berhasil dimuat tetapi kosong.
class EmptyView extends StatelessWidget {
  final String message;
  final IconData icon;

  const EmptyView({
    super.key,
    required this.message,
    this.icon = Icons.inbox_outlined,
  });

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Padding(
        padding: const EdgeInsets.all(24),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            Icon(icon, size: 64, color: Theme.of(context).colorScheme.outline),
            const SizedBox(height: 12),
            Text(message, textAlign: TextAlign.center),
          ],
        ),
      ),
    );
  }
}

/// Tampilan saat terjadi error, lengkap dengan tombol "Coba Lagi".
class ErrorView extends StatelessWidget {
  final String message;
  final VoidCallback onRetry;

  const ErrorView({super.key, required this.message, required this.onRetry});

  @override
```

```

Widget build(BuildContext context) {
  final colorScheme = Theme.of(context).colorScheme;
  return Center(
    child: Padding(
      padding: const EdgeInsets.all(24),
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          Icon(Icons.cloud_off, size: 64, color: colorScheme.error),
          const SizedBox(height: 12),
          Text(
            'Oops, terjadi kesalahan',
            style: Theme.of(context).textTheme.titleMedium,
          ),
          const SizedBox(height: 8),
          Text(message, textAlign: TextAlign.center),
          const SizedBox(height: 16),
          FilledButton.icon(
            onPressed: onRetry,
            icon: const Icon(Icons.refresh),
            label: const Text('Coba Lagi'),
          ),
        ],
      ),
    ),
  );
}

/// Banner pesan error kecil (dipakai di dalam form).
class ErrorBanner extends StatelessWidget {
  final String message;

  const ErrorBanner({super.key, required this.message});

  @override
  Widget build(BuildContext context) {
    final colorScheme = Theme.of(context).colorScheme;
    return Container(
      padding: const EdgeInsets.all(12),
      decoration: BoxDecoration(
        color: colorScheme.errorContainer,
        borderRadius: BorderRadius.circular(8),
      ),
      child: Row(
        children: [
          Icon(Icons.error_outline, color: colorScheme.onErrorContainer),
          const SizedBox(width: 8),
          Expanded(
            child: Text(
              message,
              style: TextStyle(color: colorScheme.onErrorContainer),
            ),
          ),
        ],
      ),
    );
  }
}

```

A.9 lib/widgets/menu_card.dart

```
import 'package:flutter/material.dart';

import '../models/menu_item.dart';

/// Kartu satu menu pada daftar di Home.
class MenuCard extends StatelessWidget {
  final MenuItem menuItem;
  final VoidCallback onTap;

  const MenuCard({super.key, required this.menuItem, required this.onTap});

  @override
  Widget build(BuildContext context) {
    final colorScheme = Theme.of(context).colorScheme;
    return Card(
      margin: const EdgeInsets.symmetric(horizontal: 16, vertical: 6),
      child: ListTile(
        contentPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
        leading: CircleAvatar(
          backgroundColor: colorScheme.primaryContainer,
          child: Icon(
            iconForCategory(menuItem.category),
            color: colorScheme.onPrimaryContainer,
          ),
        ),
        title: Text(menuItem.name),
        subtitle: Text('${menuItem.cafeteriaName} • ${menuItem.formattedPrice}'),
        trailing: const Icon(Icons.chevron_right),
        onTap: onTap,
      ),
    );
  }
}

/// Ikon sederhana berdasarkan kategori menu.
IconData iconForCategory(String category) {
  return switch (category) {
    'Rice' => Icons.rice_bowl,
    'Noodles' => Icons.ramen_dining,
    'Drinks' => Icons.local_cafe,
    _ => Icons.fastfood,
  };
}
```

A.10 lib/screens/login_screen.dart

```
import 'package:flutter/material.dart';

import '../data/auth_repository.dart';
import '../routes/app_routes.dart';
import '../utils/validators.dart';
import '../widgets/state_views.dart';

class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}
```

```

class _LoginScreenState extends State<LoginScreen> {
  // Kunci untuk mengakses state Form (validate, save, reset)
  final _formKey = GlobalKey<FormState>();

  // Controller untuk membaca isi TextFormField
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();

  final _authRepository = AuthRepository();

  bool _obscurePassword = true;
  bool _isLoading = false;
  String? _errorMessage;

  @override
  void dispose() {
    // WAJIB: controller dibuang saat screen ditutup agar tidak memory leak
    _emailController.dispose();
    _passwordController.dispose();
    super.dispose();
  }

  Future<void> _submit() async {
    FocusScope.of(context).unfocus(); // tutup keyboard

    // 1. Validasi semua field di dalam Form
    final isValid = _formKey.currentState?.validate() ?? false;
    if (!isValid) return;

    // 2. Masuk ke loading state
    setState(() {
      _isLoading = true;
      _errorMessage = null;
    });

    try {
      // 3. Proses async (simulasi request ke server)
      final userName = await _authRepository.login(
        email: _emailController.text.trim(),
        password: _passwordController.text,
      );

      // 4. Setelah await, pastikan screen masih ada sebelum memakai context
      if (!mounted) return;

      // 5. Sukses -> ganti Login dengan Home (Login dibuang dari stack)
      Navigator.pushReplacementNamed(
        context,
        AppRoutes.home,
        arguments: userName,
      );
    } on AuthException catch (e) {
      if (!mounted) return;
      setState(() => _errorMessage = e.message);
    } catch (e) {
      if (!mounted) return;
      setState(() => _errorMessage = 'Terjadi kesalahan. Coba lagi nanti.');
```

```

@override
Widget build(BuildContext context) {
  final textTheme = Theme.of(context).textTheme;
  final colorScheme = Theme.of(context).colorScheme;

  return Scaffold(
    body: SafeArea(
      child: Center(
        // Scroll agar tidak overflow ketika keyboard muncul
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(24),
          child: ConstrainedBox(
            constraints: const BoxConstraints(maxWidth: 420),
            child: Form(
              key: _formKey,
              autovalidateMode: AutovalidateMode.onUserInteraction,
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.stretch,
                children: [
                  Icon(Icons.restaurant_menu, size: 64, color: colorScheme.primary),
                  const SizedBox(height: 16),
                  Text(
                    'Campus Food',
                    textAlign: TextAlign.center,
                    style: textTheme.headlineMedium,
                  ),
                  const SizedBox(height: 8),
                  Text(
                    'Masuk untuk melihat menu kantin kampus',
                    textAlign: TextAlign.center,
                    style: textTheme.bodyMedium,
                  ),
                  const SizedBox(height: 32),

                  // Error dari server ditampilkan sebagai banner
                  if (_errorMessage != null) ...[
                    ErrorBanner(message: _errorMessage!),
                    const SizedBox(height: 16),
                  ],

                  TextFormField(
                    controller: _emailController,
                    enabled: !_isLoading,
                    keyboardType: TextInputType.emailAddress,
                    textInputAction: TextInputAction.next,
                    decoration: const InputDecoration(
                      labelText: 'Email',
                      hintText: 'nama@unand.ac.id',
                      prefixIcon: Icon(Icons.email_outlined),
                      border: OutlineInputBorder(),
                    ),
                    validator: Validators.email,
                  ),
                  const SizedBox(height: 16),
                  TextFormField(
                    controller: _passwordController,
                    enabled: !_isLoading,
                    obscureText: _obscurePassword,
                    textInputAction: TextInputAction.done,
                    onFieldSubmitted: (_) => _submit(),
                    decoration: InputDecoration(
                      labelText: 'Password',
                      prefixIcon: const Icon(Icons.lock_outline),

```

```

        border: const OutlineInputBorder(),
        suffixIcon: IconButton(
          tooltip: _obscurePassword ? 'Tampilkan' : 'Sembunyikan',
          icon: Icon(
            _obscurePassword
              ? Icons.visibility_outlined
              : Icons.visibility_off_outlined,
          ),
          onPressed: () => setState(
            () => _obscurePassword = !_obscurePassword,
          ),
        ),
      ),
      validator: Validators.password,
    ),
    const SizedBox(height: 24),
    FilledButton(
      // null = tombol nonaktif selama loading
      onPressed: _isLoading ? null : _submit,
      child: _isLoading
        ? const SizedBox(
            height: 20,
            width: 20,
            child: CircularProgressIndicator(strokeWidth: 2),
          )
        : const Text('Masuk'),
    ),
    const SizedBox(height: 12),
    Text(
      'Akun demo: ${AuthRepository.demoEmail} /  

      ${AuthRepository.demoPassword}',
      textAlign: TextAlign.center,
      style: textTheme.bodySmall,
    ),
  ],
),
),
),
),
),
),
),
),
);
}
}

```

A.11 lib/screens/home_screen.dart

```

import 'package:flutter/material.dart';

import '../data/menu_repository.dart';
import '../models/menu_item.dart';
import '../routes/app_routes.dart';
import '../widgets/menu_card.dart';
import '../widgets/state_views.dart';

/// Status tampilan untuk layar yang memuat data.
enum ViewStatus { loading, success, error }

class HomeScreen extends StatefulWidget {
  // Data yang DITERIMA dari LoginScreen lewat arguments
  final String userName;

```

```

const HomeScreen({super.key, required this.userName});

@override
State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  static const List<String> _categories = [
    'Semua',
    'Rice',
    'Noodles',
    'Drinks',
    'Snacks',
  ];

  final _repository = MenuRepository();

  ViewStatus _status = ViewStatus.loading;
  List<MenuItem> _menus = [];
  String _errorMessage = '';
  String _selectedCategory = 'Semua';
  bool _simulateError = false;

  @override
  void initState() {
    super.initState();
    _loadMenus();
  }

  Future<void> _loadMenus() async {
    if (_status != ViewStatus.loading) {
      setState(() => _status = ViewStatus.loading);
    }
    try {
      final menus = await _repository.fetchMenus(simulateError: _simulateError);
      if (!mounted) return;
      setState(() {
        _menus = menus;
        _status = ViewStatus.success;
      });
    } catch (e) {
      if (!mounted) return;
      setState(() {
        _errorMessage = e.toString().replaceFirst('Exception: ', '');
        _status = ViewStatus.error;
      });
    }
  }

  List<MenuItem> get _filteredMenus {
    if (_selectedCategory == 'Semua') return _menus;
    return _menus.where((menu) => menu.category == _selectedCategory).toList();
  }

  void _openDetail(MenuItem menuItem) {
    // PASSING ARGUMENT: object MenuItem dikirim ke DetailScreen
    Navigator.pushNamed(context, AppRoutes.detail, arguments: menuItem);
  }

  Future<void> _logout() async {
    // showDialog juga memakai Navigator: dialog "mengembalikan" true/false
    final confirmed = await showDialog<bool>{

```

```

context: context,
builder: (dialogContext) => AlertDialog(
  title: const Text('Keluar'),
  content: const Text('Yakin ingin keluar dari aplikasi?'),
  actions: [
    TextButton(
      onPressed: () => Navigator.pop(dialogContext, false),
      child: const Text('Batal'),
    ),
    FilledButton(
      onPressed: () => Navigator.pop(dialogContext, true),
      child: const Text('Keluar'),
    ),
  ],
),
);

if (confirmed != true || !mounted) return;

// Kembali ke Login dan HAPUS semua screen sebelumnya dari stack
Navigator.pushNamedAndRemoveUntil(context, AppRoutes.login, (route) => false);
}

void _onMenuSelected(String value) {
  if (value == 'error') {
    setState(() => _simulateError = !_simulateError);
    _loadMenus();
  } else if (value == 'logout') {
    _logout();
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Campus Food'),
      actions: [
        IconButton(
          tooltip: 'Promo',
          icon: const Icon(Icons.local_offer_outlined),
          // Sengaja memakai route yang BELUM terdaftar -> muncul halaman 404
          onPressed: () => Navigator.pushNamed(context, '/promo'),
        ),
        PopupMenuButton<String>(
          onSelected: _onMenuSelected,
          itemBuilder: (context) => [
            CheckedPopupMenuitem(
              value: 'error',
              checked: _simulateError,
              child: const Text('Simulasikan error'),
            ),
            const PopupMenuItem(value: 'logout', child: Text('Keluar')),
          ],
        ),
      ],
    ),
    body: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Padding(
          padding: const EdgeInsets.fromLTRB(16, 16, 16, 8),
          child: Text(
            'Halo, ${widget.userName}!',

```

```

        style: Theme.of(context).textTheme.titleLarge,
      ),
    ),
    SizedBox(
      height: 48,
      child: ListView.separated(
        scrollDirection: Axis.horizontal,
        padding: const EdgeInsets.symmetric(horizontal: 16),
        itemCount: _categories.length,
        separatorBuilder: (context, index) => const SizedBox(width: 8),
        itemBuilder: (context, index) {
          final category = _categories[index];
          return Center(
            child: ChoiceChip(
              label: Text(category),
              selected: _selectedCategory == category,
              onSelect: (_) => setState(() => _selectedCategory = category),
            ),
          );
        },
      ),
    ),
    const SizedBox(height: 8),
    Expanded(child: _buildContent()),
  ],
),
);
}

/// Memilih tampilan berdasarkan status: loading / error / empty / success
Widget _buildContent() {
  return switch (_status) {
    ViewStatus.loading => const LoadingView(message: 'Memuat menu...'),
    ViewStatus.error => ErrorView(message: _errorMessage, onRetry: _loadMenus),
    ViewStatus.success => _buildMenuList(),
  };
}

Widget _buildMenuList() {
  final menus = _filteredMenus;
  if (menus.isEmpty) {
    return EmptyView(
      icon: Icons.no_food_outlined,
      message: 'Belum ada menu untuk kategori "$_selectedCategory".',
    );
  }
  return ListView.builder(
    padding: const EdgeInsets.only(bottom: 16),
    itemCount: menus.length,
    itemBuilder: (context, index) {
      final menu = menus[index];
      return MenuCard(menuItem: menu, onTap: () => _openDetail(menu));
    },
  );
}
}
}

```

A.12 lib/screens/detail_screen.dart

```

import 'package:flutter/material.dart';

import '../models/menu_item.dart';

```

```

import '../models/review.dart';
import '../routes/app_routes.dart';
import '../widgets/menu_card.dart';

class DetailScreen extends StatefulWidget {
  // Data yang DITERIMA dari HomeScreen (lewat onGenerateRoute)
  final MenuItem menuItem;

  const DetailScreen({super.key, required this.menuItem});

  @override
  State<DetailScreen> createState() => _DetailScreenState();
}

class _DetailScreenState extends State<DetailScreen> {
  final List<Review> _reviews = [];

  double get _averageRating {
    if (_reviews.isEmpty) return 0;
    final total = _reviews.fold<int>(0, (sum, review) => sum + review.rating);
    return total / _reviews.length;
  }

  Future<void> _openReviewForm() async {
    // RETURNING DATA: tunggu (await) hasil dari ReviewFormScreen
    final newReview = await Navigator.pushNamed<Review>(
      context,
      AppRoutes.reviewForm,
      arguments: widget.menuItem,
    );

    // null berarti pengguna kembali tanpa mengirim review
    if (!mounted || newReview == null) return;

    setState(() => _reviews.insert(0, newReview));
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text(
          'Terima kasih, ${newReview.reviewerName}! Review berhasil ditambahkan.',
        ),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    final menu = widget.menuItem;
    final textTheme = Theme.of(context).textTheme;
    final colorScheme = Theme.of(context).colorScheme;

    return Scaffold(
      appBar: AppBar(title: Text(menu.name)),
      body: ListView(
        padding: const EdgeInsets.fromLTRB(16, 16, 16, 96),
        children: [
          AspectRatio(
            aspectRatio: 16 / 9,
            child: Container(
              decoration: BoxDecoration(
                color: colorScheme.primaryContainer,
                borderRadius: BorderRadius.circular(16),
              ),
            ),
          ),
        ],
      ),
    );
  }
}

```

```

        child: Icon(
          iconForCategory(menu.category),
          size: 96,
          color: colorScheme.onPrimaryContainer,
        ),
      ),
    ),
    const SizedBox(height: 16),
    Text(menu.name, style: textTheme.headlineSmall),
    const SizedBox(height: 4),
    Row(
      children: [
        const Icon(Icons.storefront, size: 18),
        const SizedBox(width: 4),
        Text(menu.cafeteriaName),
        const SizedBox(width: 12),
        Chip(label: Text(menu.category)),
      ],
    ),
    const SizedBox(height: 8),
    Text(
      menu.formattedPrice,
      style: textTheme.titleLarge?.copyWith(color: colorScheme.primary),
    ),
    const SizedBox(height: 8),
    Text(menu.description),
    const Divider(height: 32),
    Row(
      children: [
        Text('Review', style: textTheme.titleMedium),
        const Spacer(),
        if (_reviews.isNotEmpty) ...[
          const Icon(Icons.star, color: Colors.amber, size: 20),
          const SizedBox(width: 4),
          Text('${_averageRating.toStringAsFixed(1)} (${_reviews.length})'),
        ],
      ],
    ),
    const SizedBox(height: 8),

    // EMPTY STATE untuk review
    if (_reviews.isEmpty)
      const Padding(
        padding: EdgeInsets.symmetric(vertical: 24),
        child: Text(
          'Belum ada review. Jadilah yang pertama!',
          textAlign: TextAlign.center,
        ),
      )
    else
      ..._reviews.map(
        (review) => Card(
          child: ListTile(
            leading: CircleAvatar(
              child: Text(review.viewerName[0].toUpperCase()),
            ),
            title: Text(review.viewerName),
            subtitle: Text(review.comment),
            trailing: Text('★' * review.rating),
          ),
        ),
      ),
    ],
  ),
  floatingActionButton: FloatingActionButton.extended(

```

```

        onPressed: _openReviewForm,
        icon: const Icon(Icons.rate_review_outlined),
        label: const Text('Tulis Review'),
      ),
    );
  }
}

```

A.13 lib/screens/review_form_screen.dart

```

import 'package:flutter/material.dart';

import '../models/menu_item.dart';
import '../models/review.dart';
import '../utils/validators.dart';

class ReviewFormScreen extends StatefulWidget {
  final MenuItem menuItem;

  const ReviewFormScreen({super.key, required this.menuItem});

  @override
  State<ReviewFormScreen> createState() => _ReviewFormScreenState();
}

class _ReviewFormScreenState extends State<ReviewFormScreen> {
  final _formKey = GlobalKey<FormState>();
  final _nameController = TextEditingController();
  final _ratingController = TextEditingController();
  final _commentController = TextEditingController();

  bool _isSubmitting = false;

  bool get _hasUnsavedInput =>
    _nameController.text.isNotEmpty ||
    _ratingController.text.isNotEmpty ||
    _commentController.text.isNotEmpty;

  @override
  void dispose() {
    _nameController.dispose();
    _ratingController.dispose();
    _commentController.dispose();
    super.dispose();
  }

  Future<void> _submit() async {
    final isValid = _formKey.currentState?.validate() ?? false;
    if (!isValid) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Periksa kembali isian yang masih salah.')),
      );
      return;
    }

    setState(() => _isSubmitting = true);
    await Future.delayed(const Duration(seconds: 1)); // simulasi kirim data
    if (!mounted) return;
  }
}

```

```

final review = Review(
  id: DateTime.now().millisecondsSinceEpoch.toString(),
  menuItemId: widget.menuItem.id,
  reviewerName: _nameController.text.trim(),
  rating: int.parse(_ratingController.text.trim()),
  comment: _commentController.text.trim(),
);

// RETURNING DATA: tutup screen ini sambil membawa object Review
Navigator.pop(context, review);
}

Future<bool> _confirmDiscard() async {
  final result = await showDialog<bool>(
    context: context,
    builder: (dialogContext) => AlertDialog(
      title: const Text('Buang review?'),
      content: const Text('Isian review belum dikirim. Yakin ingin keluar?'),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(dialogContext, false),
          child: const Text('Batal'),
        ),
        FilledButton(
          onPressed: () => Navigator.pop(dialogContext, true),
          child: const Text('Keluar'),
        ),
      ],
    ),
  );
  return result ?? false;
}

@override
Widget build(BuildContext context) {
  // PopScope: mencegah pengguna tidak sengaja keluar saat form sudah diisi
  return PopScope<Object?>{
    canPop: !_hasUnsavedInput || !_isSubmitting,
    onPopInvokedWithResult: (didPop, result) async {
      if (didPop) return; // sudah keluar, tidak perlu apa-apa
      final shouldLeave = await _confirmDiscard();
      if (shouldLeave && mounted) Navigator.pop(context);
    },
    child: Scaffold(
      appBar: AppBar(title: const Text('Tulis Review')),
      body: SafeArea(
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(16),
          child: Form(
            key: _formKey,
            autovalidateMode: AutovalidateMode.onUserInteraction,
            // Rebuild setiap ada ketikan agar canPop selalu ter-update
            onChanged: () => setState(() {}),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                // Menampilkan argument yang diterima dari DetailScreen
                Card(
                  child: ListTile(
                    leading: const Icon(Icons.fastfood),
                    title: Text(widget.menuItem.name),
                    subtitle: Text(widget.menuItem.cafeteriaName),
                  ),
                ),
                const SizedBox(height: 16),

```

```

TextFormField(
  controller: _nameController,
  enabled: !_isSubmitting,
  textCapitalization: TextCapitalization.words,
  textInputAction: TextInputAction.next,
  decoration: const InputDecoration(
    labelText: 'Nama',
    prefixIcon: Icon(Icons.person_outline),
    border: OutlineInputBorder(),
  ),
  validator: (value) =>
    Validators.minLength(value, 3, fieldName: 'Nama'),
),
const SizedBox(height: 16),
TextFormField(
  controller: _ratingController,
  enabled: !_isSubmitting,
  keyboardType: TextInputType.number,
  textInputAction: TextInputAction.next,
  decoration: const InputDecoration(
    labelText: 'Rating (1-5)',
    prefixIcon: Icon(Icons.star_outline),
    border: OutlineInputBorder(),
  ),
  validator: Validators.rating,
),
const SizedBox(height: 16),
TextFormField(
  controller: _commentController,
  enabled: !_isSubmitting,
  maxLines: 4,
  maxLength: 200,
  decoration: const InputDecoration(
    labelText: 'Komentar',
    alignLabelWithHint: true,
    border: OutlineInputBorder(),
  ),
  validator: (value) =>
    Validators.minLength(value, 10, fieldName: 'Komentar'),
),
const SizedBox(height: 16),
FilledButton.icon(
  onPressed: _isSubmitting ? null : _submit,
  icon: _isSubmitting
    ? const SizedBox(
        height: 18,
        width: 18,
        child: CircularProgressIndicator(strokeWidth: 2),
      )
    : const Icon(Icons.send),
  label: Text(_isSubmitting ? 'Mengirim...' : 'Kirim Review'),
),
),
),
),
),
),
),
);
}
}

```

A.14 lib/screens/not_found_screen.dart

```
import 'package:flutter/material.dart';

import '../routes/app_routes.dart';

/// Halaman cadangan ketika route tidak dikenal atau argument tidak valid.
class NotFoundScreen extends StatelessWidget {
  final String? routeName;
  final String message;

  const NotFoundScreen({
    super.key,
    this.routeName,
    this.message = 'Halaman yang kamu tuju belum tersedia.',
  });

  @override
  Widget build(BuildContext context) {
    final textTheme = Theme.of(context).textTheme;
    return Scaffold(
      appBar: AppBar(title: const Text('Halaman Tidak Ditemukan')),
      body: Center(
        child: Padding(
          padding: const EdgeInsets.all(24),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.min,
            children: [
              const Icon(Icons.search_off, size: 72),
              const SizedBox(height: 8),
              Text('404', style: textTheme.displaySmall),
              const SizedBox(height: 8),
              Text(message, textAlign: TextAlign.center),
              if (routeName != null) ...[
                const SizedBox(height: 4),
                Text('Route: $routeName', style: textTheme.bodySmall),
              ],
              const SizedBox(height: 16),
              FilledButton(
                onPressed: () {
                  if (Navigator.canPop(context)) {
                    Navigator.pop(context);
                  } else {
                    Navigator.pushReplacementNamed(context, AppRoutes.login);
                  }
                },
                child: const Text('Kembali'),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```